DSA STUDY BLUEPRINT SERIES

104. Introduction to Graphs

Graph Formats: Adjacency Matrices and Adjacency Lists

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Adjacency Matrix:** $V imes V$ boolean grid. O(V^2) space.
- **Adjacency List:** Array of vectors. O(V+E) space. Highly ideal for sparse graphs.

Memory & Execution Blueprint

Adjacency List format: [Node 0] → [1, 4] | [Node 1] → [0, 2, 3]

C++ Implementation Reference

Standard code layout and syntax

```
int V = 5;
vector> adj(V);
// Undirected Edge: u <-> v
adj[u].push_back(v);
adj[v].push_back(u);
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Matrix Space	$O(V^2)$
List Space	$O(V + E)$

Key Practices & Gotchas

- **Important:** Adjacency lists are highly popular due to efficient memory footprints.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.